

Explain BFS Dry Run

Let's understand the **complete logic** first, then do a proper dry run.

This problem has two main tasks:

1. Use **BFS** to find all friends exactly at the given **level**.
2. Collect the videos watched by those friends, count frequencies, and sort them.

1. What does the BFS part do?

Imagine the friendship graph:

```

Person 0
├── Person 1
└── Person 2

Person 1
└── Person 3

Person 2
└── Person 4
  
```

If:

```

C++
id = 0
level = 2
  
```

Then BFS levels are:

```

Level 0 → [0]
Level 1 → [1, 2]
Level 2 → [3, 4]
  
```

We only want:

```

[3, 4]
  
```

The important idea is:

After processing exactly **level** BFS layers, the queue contains the people at that exact level.

Example for dry run

Let's use this example:

```

C++
watchedVideos = {
    {"A", "B"}, // person 0
    {"C"},      // person 1
    {"B", "C"}, // person 2
    {"D", "B"}, // person 3
    {"C", "D"}  // person 4
};

friends = {
    {1, 2}, // person 0
    {0, 3}, // person 1
    {0, 4}, // person 2
    {1},    // person 3
    {2}     // person 4
};

id = 0;
level = 2;
  
```

Graph:



We need videos watched by people at **level 2 from person 0**.

So ultimately we need:

```

Person 3 → D, B
Person 4 → C, D
  
```

Step 1: Initialize BFS

```
C++
int n = friends.size();
```

There are:

```
n = 5
```

Now:

```
C++
queue<int> q;
vector<bool> visited(n, false);
```

Initially:

```
Queue:
[]
visited:
[false, false, false, false, false]
```

Push the starting person:

```
C++
q.push(id);
visited[id] = true;
```

Since:

```
C++
id = 0
```

We get:

```
Queue:
[0]
visited:
[true, false, false, false, false]
currentLevel = 0
```

Step 2: BFS while we haven't reached the target level

```
C++
while (!q.empty() && currentLevel < level)
```

Currently:

```
currentLevel = 0
level = 2
```

So:

```
0 < 2 → true
```

Enter BFS.

BFS Level 0

```
C++
int size = q.size();
```

Queue contains:

```
[0]
```

Therefore:

```
size = 1
```

This is extremely important.

```
C++
size
```

tells us:

How many nodes belong to the current BFS level?

Now:

```
C++
while (size--)
```

We process person 0.

Before processing

```
Queue:
[0]

visited:
[true, false, false, false, false]
```

Take:

```
C++
int person = q.front();
q.pop();
```

So:

```
person = 0

Queue:
[]
```

Now check all friends of person 0:

```
C++
friends[0] = {1, 2}
```

Friend 1

```
C++
if (!visited[1])
```

Currently:

```
visited[1] = false
```

So:

```
C++
visited[1] = true;
q.push(1);
```

Now:

```
Queue:
[1]

visited:
[true, true, false, false, false]
```

Friend 2

Similarly:

```
C++
visited[2] = true;
q.push(2);
```

Now:

```
Queue:
[1, 2]
```

```
visited:
[true, true, true, false, false]
```

Finished processing all nodes at level 0.

Then:

```
C++
currentLevel++;
```

So:

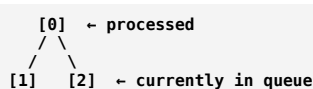
```
currentLevel = 1
```

Current situation

```
Processed:
Level 0 → [0]

Queue now contains:
Level 1 → [1, 2]
```

Visualization:



● BFS Level 1

Check condition:

```
currentLevel = 1
level = 2
1 < 2 → true
```

Enter again.

```
C++
int size = q.size();
```

Queue:

```
[1, 2]
```

Therefore:

```
size = 2
```

This means:

Process exactly these 2 people.

Process Person 1

```
C++
person = q.front();
q.pop();
```

So:

```
person = 1

Queue:
[2]
```

Friends:

```
C++
friends[1] = {0, 3}
```

Friend 0

```
visited[0] = true
```

Already visited.

So ignore.

Why?

Because otherwise we could go:

```
0 → 1 → 0 → 1 → ...
```

and keep revisiting nodes.

Friend 3

```
visited[3] = false
```

So:

```
C++
visited[3] = true;
q.push(3);
```

Queue:

```
[2, 3]
```

Visited:

```
[true, true, true, true, false]
```

Process Person 2

Next:

```
C++
person = q.front();
q.pop();
```

So:

```
person = 2

Queue:
[3]
```

Friends:

```
C++
friends[2] = {0, 4}
```

Friend 0

Already visited.

Ignore.

Friend 4

Not visited.

So:

```
C++
visited[4] = true;
q.push(4);
```

Queue:

```
[3, 4]
```

Visited:

```
[true, true, true, true, true]
```

Now we have finished processing the original:

```
size = 2
```

nodes.

Increment:

```
C++
currentLevel++;
```

Now:

```
currentLevel = 2
```

🚩 This is the key moment

Now check:

```
C++
while (!q.empty() && currentLevel < level)
```

We have:

```
currentLevel = 2
level = 2
2 < 2 → false
```

So BFS stops.

And the queue contains:

```
Queue:
[3, 4]
```

These are exactly the people at:

```
Level 2
```

So this comment is correct:

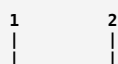
```
C++
// Now queue contains people exactly at 'level'
```

Graph now looks like:

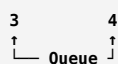
Level 0



Level 1



Level 2



Step 3: Count videos

Now:

```
C++
unordered_map<string, int> freq;
```

Initially:

```
freq = {}
```

Queue:

```
[3, 4]
```

Process Person 3

```
C++
person = q.front();
q.pop();
```

```
person = 3
Queue = [4]
```

Videos:

```
C++
watchedVideos[3] = {"D", "B"}
```

Loop:

```
C++
for (string video : watchedVideos[person])
```

Video D

```
C++
freq["D"]++;
```

Map:

```
D → 1
```

Video B

```
C++
freq["B"]++;
```

Map:

```
D → 1
B → 1
```

Process Person 4

```
person = 4
Queue:
[]
```

Videos:

```
C++
watchedVideos[4] = {"C", "D"}
```

Video C

```
C → 1
```

Map:

```
D → 1
B → 1
C → 1
```

Video D

D already exists:

```
D → 1
```

After:

```
C++
freq["D"]++;
```

We get:

```
D → 2
B → 1
C → 1
```

Final frequency map:

Video	Frequency
B	1
C	1
D	2

Step 4: Convert map into vector

The code:

```
C++
vector<pair<string, int>> videos;

for (auto& it : freq) {
    videos.push_back({it.first, it.second});
}
```

Creates something like:

```
videos = [
    {"D", 2},
    {"B", 1},
    {"C", 1}
]
```

Important:

`unordered_map` does **not guarantee any order**.

So the initial order could be different.

For example:

```
{"C", 1}, {"D", 2}, {"B", 1}
```

That's why we need sorting.

Step 5: Sorting

The code:

```
C++
sort(videos.begin(), videos.end(),
    [](auto& a, auto& b) {
        if (a.second != b.second) {
            return a.second < b.second;
        }
        return a.first < b.first;
    });
```

Each pair is:

```
{videoName, frequency}
```

For example:

```
{"D", 2}
```

means:

```
video = D
frequency = 2
```

Sorting rule 1: Smaller frequency first

```
C++
```



```
if (a.second != b.second) {
    return a.second < b.second;
}
```

So:

```
B → 1
C → 1
D → 2
```

Frequency 1 comes before frequency 2.

Sorting rule 2: If frequency is same → alphabetical order

We have:

```
B → 1
C → 1
```

Same frequency.

So:

```
C++
return a.first < b.first;
```

Compare:

```
"B" < "C"
```

Therefore:

```
B comes before C
```

Final sorted vector:

```
videos = [
    {"B", 1},
    {"C", 1},
    {"D", 2}
]
```

Step 6: Create the answer

```
C++
vector<string> ans;
for (auto& video : videos) {
    ans.push_back(video.first);
}
```

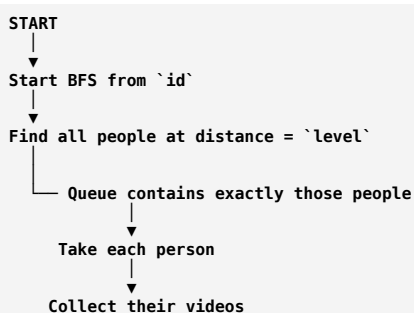
Iteration:

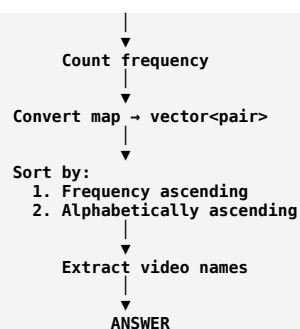
```
{"B", 1} → push "B"
{"C", 1} → push "C"
{"D", 2} → push "D"
```

Final answer:

```
C++
["B", "C", "D"]
```

Complete flow of the algorithm





The core intuition in one sentence

BFS finds WHO we need → Map counts WHAT they watched → Sorting decides the required order.

Or even more precisely:

```

id
↓
BFS
↓
People exactly at level `k`
↓
Collect all their videos
↓
Count each video
↓
Sort by frequency, then name
↓
Answer
  
```

The **most important part** to understand is this:

```

C++
int size = q.size();
while (size-- > 0) {
    ...
}
  
```

This ensures we process **only one BFS level at a time**. After processing `level` layers, the remaining queue contains exactly the friends at the required distance.

ChatGPT can make mistakes. Check important info.